



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación
Asignatura: Desarrollo de Aplicaciones Móviles
NRC: 30738

Tarea 1

Creación de una Aplicación Móvil B

Integrantes:

Cáceres Germán
Caiza Carlos
Campoverde Pablo
Mortensen Eduardo
Zurita Pablo

Docente: Ing. Doris Chicacza MSc

Fecha: 29 de abril de 2026

Link Entregables:

<https://github.com/Gpcaceres/grupo-moviles-poryectos.git>

Índice

Índice de Algoritmos	ii
Índice de Anexos	ii
1 Introducción	1
1.1 Objetivos	1
1.1.1 Objetivo General	1
1.1.2 Objetivos Específicos	1
2 Marco Teórico	1
2.1 Flutter y Dart	1
2.2 Patrón Arquitectónico MVC	2
2.3 Diseño Atómico (<i>Atomic Design</i>)	2
2.4 Contraste: MVC versus Diseño Atómico	3
2.5 Gestión de Estado con ChangeNotifier	3
3 Metodología	3
3.1 Herramientas y Tecnologías Utilizadas	4
3.2 Estructura del Proyecto	4
4 Desarrollo	4
4.1 Ejercicio 5 – Conversión de Medidas	4
4.1.1 Descripción	4
4.1.2 Diagrama de Flujo	5
4.1.3 Pseudocódigo	6
4.1.4 Implementación MVC	6
4.2 Ejercicio 6 – Capacidad de Disco Duro	6
4.2.1 Descripción	6
4.2.2 Diagrama de Flujo	7
4.2.3 Pseudocódigo	7
4.2.4 Implementación MVC	7
4.3 Ejercicio 8 – Compañía de Seguros	8
4.3.1 Descripción	8
4.3.2 Diagrama de Flujo	8
4.3.3 Pseudocódigo	9
4.3.4 Implementación MVC	9
4.4 Ejercicio 9 – Producción de Operario	9
4.4.1 Descripción	9
4.4.2 Diagrama de Flujo	10
4.4.3 Pseudocódigo	11
4.4.4 Implementación MVC	11
4.5 Ejercicio 10 – Supermercado	11
4.5.1 Descripción	11
4.5.2 Diagrama de Flujo	12
4.5.3 Pseudocódigo	12
4.5.4 Implementación MVC	13

5 Conclusiones	13
6 Recomendaciones	13
Referencias	15
Anexos	16

Índice de figuras

1 Diagrama de flujo – Ejercicio 5: Conversión de Medidas	5
2 Diagrama de flujo – Ejercicio 6: Capacidad de Disco Duro	7
3 Diagrama de flujo – Ejercicio 8: Compañía de Seguros	8
4 Diagrama de flujo – Ejercicio 9: Producción de Operario	10
5 Diagrama de flujo – Ejercicio 10: Descuento en Supermercado	12

Índice de Algoritmos

1 Conversión de Medidas	6
2 Conversión de Capacidad de Almacenamiento	7
3 Cálculo de Tarifa de Seguro	9
4 Cálculo de Incentivo de Operario	11
5 Descuento en Supermercado	12

Índice de Anexos

1 Menú principal de la aplicación	16
2 Ejercicio 5 – Conversión de Medidas	16
3 Ejercicio 6 – Capacidad de Disco Duro	17
4 Ejercicio 8 – Compañía de Seguros	17
5 Ejercicio 9 – Producción de Operario	18
6 Ejercicio 10 – Supermercado	18

1. Introducción

El desarrollo de aplicaciones móviles representa uno de los campos de mayor crecimiento en la industria del software contemporáneo. En este contexto, el presente documento constituye el informe técnico de la **Tarea 1 – Creación de una Aplicación Móvil B**, desarrollada en la asignatura *Desarrollo de Aplicaciones Móviles* de la Universidad de las Fuerzas Armadas ESPE.

La aplicación, denominada **Deber 1 – Grupo Móviles**, fue construida con el *framework* **Flutter** y el lenguaje de programación **Dart**, adoptando el patrón arquitectónico **Modelo–Vista–Controlador (MVC)** y contrastando sus principios con los del **Diseño Atómico**. La solución integra cinco ejercicios prácticos de cálculo accesibles desde un menú principal centralizado.

1.1 Objetivos

1.1.1 Objetivo General

Desarrollar una aplicación móvil multiplataforma con Flutter/Dart que implemente el patrón MVC para resolver cinco problemas de cálculo, evaluando la calidad del código, la organización arquitectónica y la experiencia de usuario.

1.1.2 Objetivos Específicos

- Aplicar el patrón MVC para separar las responsabilidades de datos, lógica y presentación.
- Contrastar la arquitectura MVC con los principios del Diseño Atómico en el contexto de Flutter.
- Implementar la lógica de negocio de cinco ejercicios de cálculo como clases Dart puras.
- Garantizar una interfaz de usuario *responsive* e intuitiva mediante el árbol de widgets de Flutter.
- Documentar el flujo lógico de cada ejercicio mediante diagramas de flujo y pseudocódigo.

2. Marco Teórico

2.1 Flutter y Dart

Flutter es un *framework* de código abierto creado por Google para el desarrollo de aplicaciones nativas multiplataforma (iOS, Android, Web, Desktop) a partir de una única base de código. Utiliza el lenguaje de programación Dart y un motor de renderizado propio (*Skia/Impeller*) que no depende de componentes nativos del sistema operativo, lo que garantiza una apariencia visual consistente entre plataformas [1].

Dart es un lenguaje orientado a objetos, fuertemente tipado y compilado, diseñado para el desarrollo de clientes de alto rendimiento. Soporta programación asíncrona mediante `async/await` y la compilación *Ahead-of-Time* (AOT) que garantiza tiempos de inicio rápidos en producción [2]. Entre sus características más relevantes se destacan:

- **Tipado estático optativo:** permite detectar errores en tiempo de compilación sin perder la flexibilidad del tipado dinámico.

- **Null safety:** el sistema de tipos garantiza que las variables no puedan ser `null` a menos que se declare explícitamente, reduciendo los errores en tiempo de ejecución.
- **Compilación AOT y JIT:** AOT para producción (rendimiento) y JIT para desarrollo (*Hot Reload*).

Entre las características principales de Flutter se destacan:

- **Hot Reload:** permite visualizar cambios de código en tiempo real sin reiniciar la aplicación, acelerando el ciclo de desarrollo.
- **Árbol de widgets:** toda la UI se construye mediante widgets composables, inmutables y reactivos.
- **Rendimiento nativo:** el código se compila a ARM o x86, evitando puentes JavaScript como en React Native.

2.2 Patrón Arquitectónico MVC

El patrón **Modelo–Vista–Controlador (MVC)** es un paradigma de diseño de software que separa la aplicación en tres capas con responsabilidades bien definidas, lo que facilita el mantenimiento, la escalabilidad y la prueba unitaria del código [3]:

- **Modelo (*Model*):** encapsula los datos y la lógica de negocio. Es completamente independiente de la capa de presentación y puede ser probado sin necesidad de una interfaz gráfica.
- **Vista (*View*):** es responsable de la presentación de la información al usuario. Solo muestra datos, no los procesa ni almacena.
- **Controlador (*Controller*):** actúa como intermediario entre el Modelo y la Vista. Recibe las entradas del usuario, las delega al Modelo y actualiza la Vista con los resultados.

En Flutter, el MVC se implementa organizando las carpetas `model/`, `view/` y `controller/` dentro de `lib/`. Los modelos son clases Dart puras, los controladores extienden `ChangeNotifier` para notificar cambios reactivos, y las vistas son `StatefulWidget` o `StatelessWidget` que consumen dichos controladores.

2.3 Diseño Atómico (*Atomic Design*)

El **Diseño Atómico**, propuesto por Brad Frost [4], es una metodología de diseño de interfaces que aplica una analogía con la química para organizar los componentes de UI en cinco niveles de composición:

1. **Átomos:** los componentes mínimos e indivisibles. En Flutter: un `TextField`, un `ElevatedButton`, un `Text`.
2. **Moléculas:** combinaciones de átomos con una función específica. Ejemplo: un campo de entrada con etiqueta y botón de envío.
3. **Organismos:** grupos de moléculas que forman una sección completa de la interfaz. Ejemplo: un formulario con campos, validación y resultado.
4. **Plantillas (*Templates*):** definen la estructura general de la página sin contenido real; son el esqueleto de la pantalla.
5. **Páginas (*Pages*):** instancias concretas de plantillas con contenido real y navegación activa.

2.4 Contraste: MVC versus Diseño Atómico

Aunque MVC y Diseño Atómico abordan la organización del software desde perspectivas distintas, son paradigmas complementarios que operan en niveles de abstracción diferentes.

Cuadro 1: Comparativa entre MVC y Diseño Atómico en el contexto de Flutter

Criterio	MVC	Diseño Atómico
Enfoque	Separación de capas lógicas (datos, lógica, presentación)	Composición jerárquica de componentes de UI
Nivel de abstracción	Arquitectura completa de la aplicación	Organización interna de la capa de presentación
Unidad base	Módulo funcional (modelo/controller/view)	Widget atómico
Reutilización	Por capa y por función	Por nivel de composición
Estructura en Flutter	Carpetas model/, view/, controller/	Átomos, moléculas y organismos dentro de cada vista
Gestión de estado	En el Controlador (ChangeNotifier)	En el Organismo o Página (widget padre)

En la aplicación desarrollada, **MVC** rige la organización de archivos y la separación de responsabilidades a nivel de carpetas, mientras que el **Diseño Atómico** guía la composición interna de cada Vista: los `TextField` y `ElevatedButton` son **átomos**; los bloques de entrada con su etiqueta son **moléculas**; cada pantalla de ejercicio (formulario + resultado) es un **organismo**; y el menú principal actúa como **página**.

2.5 Gestión de Estado con ChangeNotifier

Flutter provee el mecanismo `ChangeNotifier` para gestionar el estado de la aplicación siguiendo el principio de separación de responsabilidades del MVC [5]. Los controladores extienden `ChangeNotifier` y llaman a `notifyListeners()` cuando los datos del modelo cambian; las vistas se suscriben mediante `ListenableBuilder` o `setState()` para reconstruirse automáticamente.

3. Metodología

El desarrollo de la aplicación siguió un proceso iterativo e incremental organizado en las siguientes etapas:

1. **Análisis de requisitos:** identificación de los cinco ejercicios de cálculo, sus entradas, salidas y reglas de negocio.
2. **Diseño arquitectónico:** definición de la estructura MVC y la organización de carpetas dentro de `lib/`.
3. **Modelado:** creación de las clases de modelo con lógica de cálculo pura en Dart, sin dependencias de Flutter.

4. **Implementación de controladores:** desarrollo de los controladores que gestionan el estado y median entre el modelo y la vista.
5. **Diseño de vistas:** construcción de las interfaces con Flutter aplicando principios del Diseño Atómico.
6. **Integración:** creación del menú principal (**MenuView**) como punto de entrada unificado a todos los ejercicios.
7. **Pruebas:** verificación manual del comportamiento en emulador Android.

3.1 Herramientas y Tecnologías Utilizadas

- **Flutter SDK 3.x:** *framework* de desarrollo multiplataforma.
- **Dart 3.x:** lenguaje de programación con *null safety*.
- **Visual Studio Code** con extensiones Flutter y Dart.
- **Android Studio / Android Emulator:** entorno de emulación.
- **Git / GitHub:** control de versiones y colaboración.

3.2 Estructura del Proyecto

```
lib/  
  main.dart                <- Punto de entrada  
  model/  
    medidas_model.dart  
    storage_model.dart  
    customer_model.dart  
    incentivo_model.dart  
    supermercado_model.dart  
  controller/  
    medidas_controller.dart  
    storage_controller.dart  
    finance_controller.dart  
    incentivo_controller.dart  
    supermercado_controller.dart  
  view/  
    menu_view.dart  
    medidas_view.dart  
    storage_view.dart  
    view_finance.dart  
    incentivo_view.dart  
    supermercado_view.dart
```

4. Desarrollo

4.1 Ejercicio 5 – Conversión de Medidas

4.1.1 Descripción

El usuario ingresa una distancia en metros. La aplicación calcula y muestra los equivalentes en centímetros, pulgadas, pies y yardas.

Fórmulas de conversión:

$$\text{centímetros} = \text{metros} \times 100 \quad (1)$$

$$\text{pulgadas} = \frac{\text{centímetros}}{2,54} \quad (2)$$

$$\text{pies} = \frac{\text{pulgadas}}{12} \quad (3)$$

$$\text{yardas} = \frac{\text{pies}}{3} \quad (4)$$

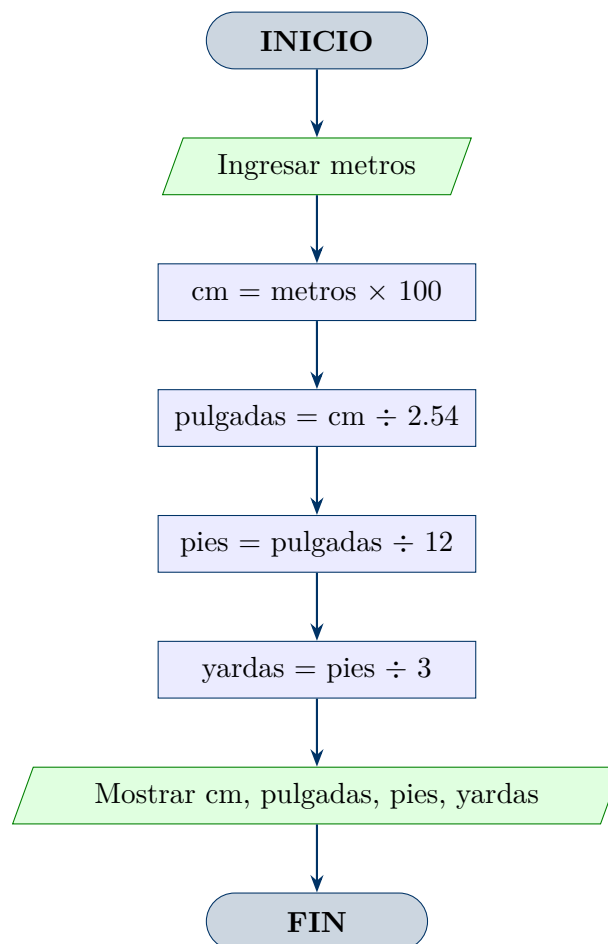
4.1.2 Diagrama de Flujo

Figura 1: Diagrama de flujo – Ejercicio 5: Conversión de Medidas

4.1.3 Pseudocódigo

Algoritmo 1 Conversión de Medidas

Require: $\text{metros} \geq 0$

Ensure: *centimetros, pulgadas, pies, yardas*

- 1: **LEER** *metros*
 - 2: $\text{centimetros} \leftarrow \text{metros} \times 100$
 - 3: $\text{pulgadas} \leftarrow \text{centimetros} \div 2,54$
 - 4: $\text{pies} \leftarrow \text{pulgadas} \div 12$
 - 5: $\text{yardas} \leftarrow \text{pies} \div 3$
 - 6: **MOSTRAR** *centimetros, pulgadas, pies, yardas*
-

4.1.4 Implementación MVC

- **Modelo** (*medidas_model.dart*): método estático `calcular(metros)` que retorna un objeto `MedidasModel` con todos los valores calculados como atributos finales.
- **Controlador** (*medidas_controller.dart*): recibe la entrada del usuario desde la vista y delega el cálculo al modelo.
- **Vista** (*medidas_view.dart*): `StatefulWidget` con un `TextField` de entrada y widgets `Text` para cada resultado.

4.2 Ejercicio 6 – Capacidad de Disco Duro

4.2.1 Descripción

El usuario ingresa la capacidad de almacenamiento en gigabytes (GB). La aplicación convierte el valor a megabytes (MB), kilobytes (KB) y bytes (B) usando las equivalencias binarias estándar.

Fórmulas:

$$\text{MB} = \text{GB} \times 1024 \quad (5)$$

$$\text{KB} = \text{MB} \times 1024 \quad (6)$$

$$\text{Bytes} = \text{KB} \times 1024 \quad (7)$$

4.2.2 Diagrama de Flujo

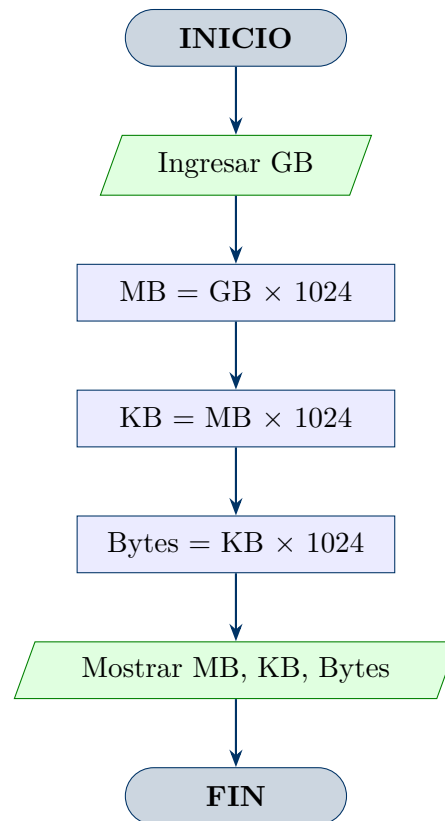


Figura 2: Diagrama de flujo – Ejercicio 6: Capacidad de Disco Duro

4.2.3 Pseudocódigo

Algoritmo 2 Conversión de Capacidad de Almacenamiento

Require: $GB \geq 0$

Ensure: $MB, KB, Bytes$

- 1: **LEER** GB
 - 2: $MB \leftarrow GB \times 1024$
 - 3: $KB \leftarrow MB \times 1024$
 - 4: $Bytes \leftarrow KB \times 1024$
 - 5: **MOSTRAR** $MB, KB, Bytes$
-

4.2.4 Implementación MVC

- **Modelo** (`storage_model.dart`): clase con atributo mutable `gigabytes` y *getters* computados para `megabytes`, `kilobytes` y `bytes`.
- **Controlador** (`storage_controller.dart`): extiende `ChangeNotifier`; el método `updateGigabytes()` actualiza el modelo y llama a `notifyListeners()`.
- **Vista** (`storage_view.dart`): `StatefulWidget` que escucha cambios del controlador y actualiza los valores mostrados de forma reactiva.

4.3 Ejercicio 8 – Compañía de Seguros

4.3.1 Descripción

Una compañía de seguros calcula la tarifa mensual de un cliente según el monto asegurado. Si el monto es menor a \$50 000, la tarifa es el 3 %; si es mayor o igual, es el 2 %.

Regla de negocio:

$$\text{tarifa} = \begin{cases} \text{monto} \times 0,03 & \text{si } \text{monto} < 50\,000 \\ \text{monto} \times 0,02 & \text{si } \text{monto} \geq 50\,000 \end{cases}$$

4.3.2 Diagrama de Flujo

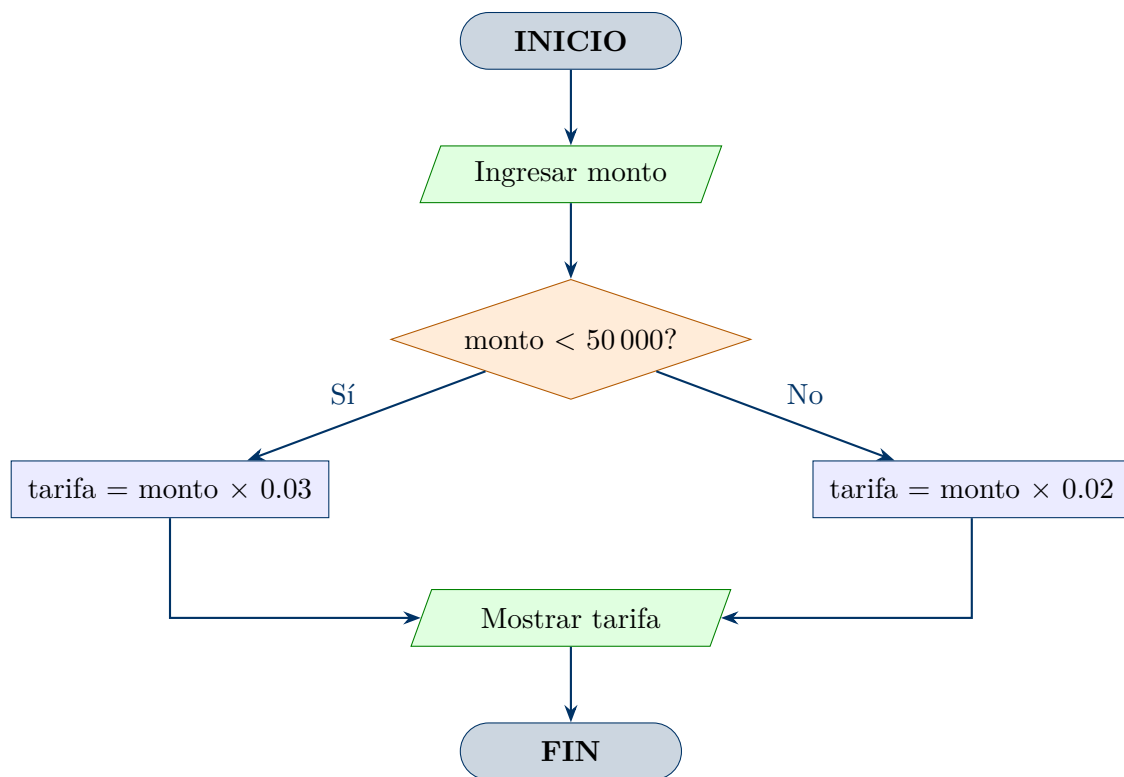


Figura 3: Diagrama de flujo – Ejercicio 8: Compañía de Seguros

4.3.3 Pseudocódigo

Algoritmo 3 Cálculo de Tarifa de Seguro

Require: *monto* > 0

Ensure: *tarifa*

```

1: LEER monto
2: if monto < 50000 then
3:   tarifa ← monto × 0,03
4: else
5:   tarifa ← monto × 0,02
6: end if
7: MOSTRAR tarifa

```

4.3.4 Implementación MVC

- **Modelo** (*customer_model.dart*): clase **CustomerFinance** con atributos **amount** (monto asegurado) y **fee** (tarifa calculada).
- **Controlador** (*finance_controller.dart*): método **calculateFee(customer)** aplica la lógica condicional y asigna el valor a **customer.fee**.
- **Vista** (*view_finance.dart*): formulario con campo de ingreso del monto y área de visualización de la tarifa calculada.

4.4 Ejercicio 9 – Producción de Operario

4.4.1 Descripción

Un operario registra su producción diaria de lunes a sábado (6 días). Se calcula el promedio semanal. Si el promedio es mayor o igual a 100 unidades, el operario recibe incentivos; en caso contrario, no.

Fórmulas:

$$\text{promedio} = \frac{\sum_{i=1}^6 p_i}{6}, \quad \text{incentivo} = \begin{cases} \text{Sí} & \text{si promedio} \geq 100 \\ \text{No} & \text{si promedio} < 100 \end{cases}$$

4.4.2 Diagrama de Flujo

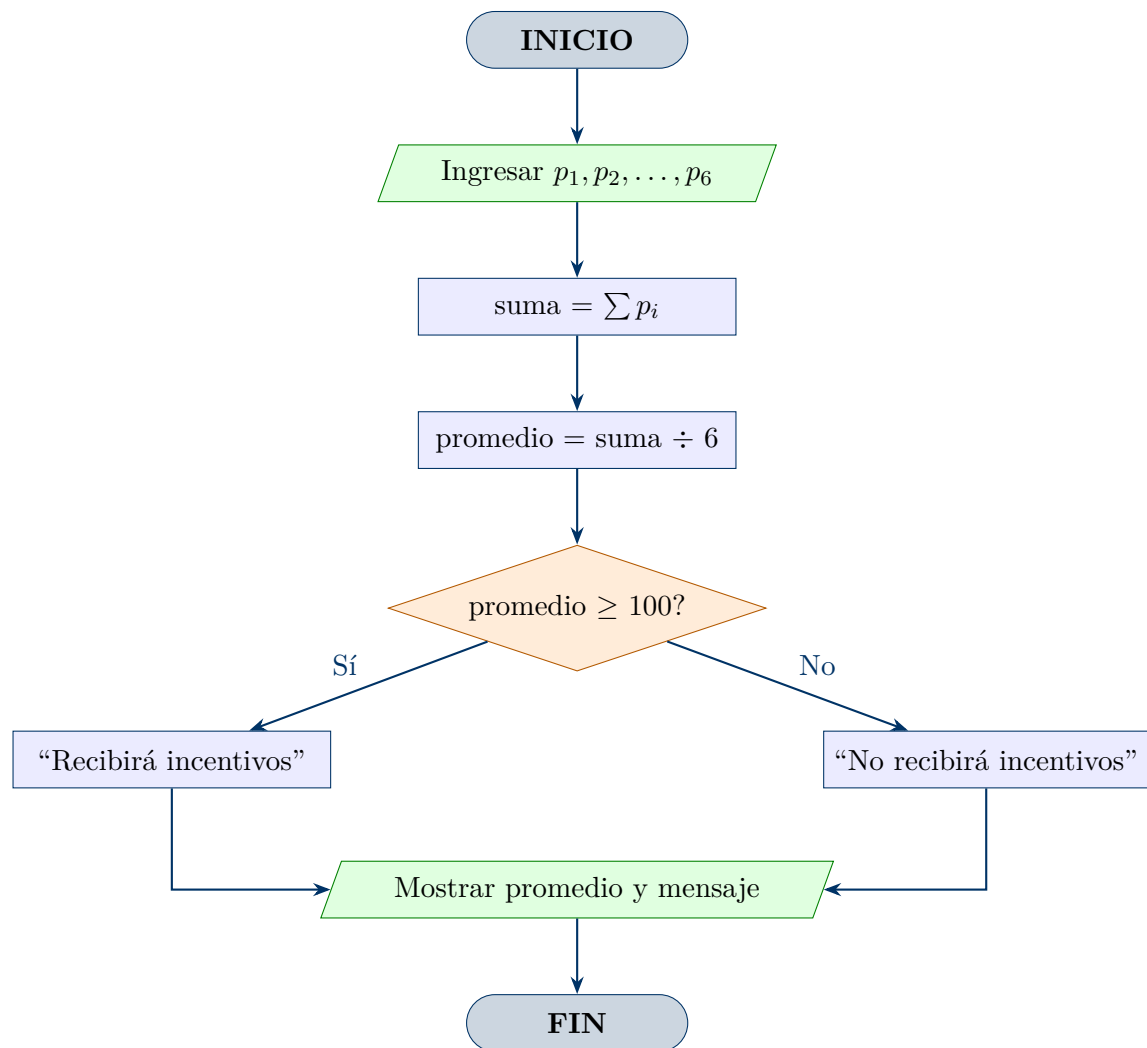


Figura 4: Diagrama de flujo – Ejercicio 9: Producción de Operario

4.4.3 Pseudocódigo

Algoritmo 4 Cálculo de Incentivo de Operario

Require: $p_i \geq 0$ para $i = 1 \dots 6$

Ensure: *promedio*, *resultado*

```

1: suma  $\leftarrow 0$ 
2: for  $i \leftarrow 1$  hasta 6 do
3:   LEER  $p_i$ 
4:    $\textit{suma} \leftarrow \textit{suma} + p_i$ 
5: end for
6:  $\textit{promedio} \leftarrow \textit{suma} \div 6$ 
7: if  $\textit{promedio} \geq 100$  then
8:    $\textit{resultado} \leftarrow \text{“Recibirá incentivos”}$ 
9: else
10:   $\textit{resultado} \leftarrow \text{“No recibirá incentivos”}$ 
11: end if
12: MOSTRAR promedio, resultado

```

4.4.4 Implementación MVC

- **Modelo** (*incentivo_model.dart*): métodos estáticos `promedio(List<int>)` y `recibeIncentivo(List<int>)` que operan sobre la lista de producciones.
- **Controlador** (*incentivo_controller.dart*): método `verificar(List<String>)` valida y convierte las cadenas de texto, invoca al modelo y retorna el mensaje formateado.
- **Vista** (*incentivo_view.dart*): seis campos de texto (uno por día laboral) y un botón que dispara el cálculo.

4.5 Ejercicio 10 – Supermercado

4.5.1 Descripción

Un supermercado aplica descuentos aleatorios a sus clientes. El cliente ingresa el total de su compra; la aplicación genera (o acepta) un número al azar entre 1 y 100. Si el número es menor a 74, el descuento es del 15 %; de lo contrario, es del 20 %.

Regla de negocio:

$$\text{descuento} = \begin{cases} 15 \% & \text{si } n < 74 \\ 20 \% & \text{si } n \geq 74 \end{cases}, \quad \text{total a pagar} = \text{totalCompra} \times (1 - \text{descuento})$$

4.5.2 Diagrama de Flujo

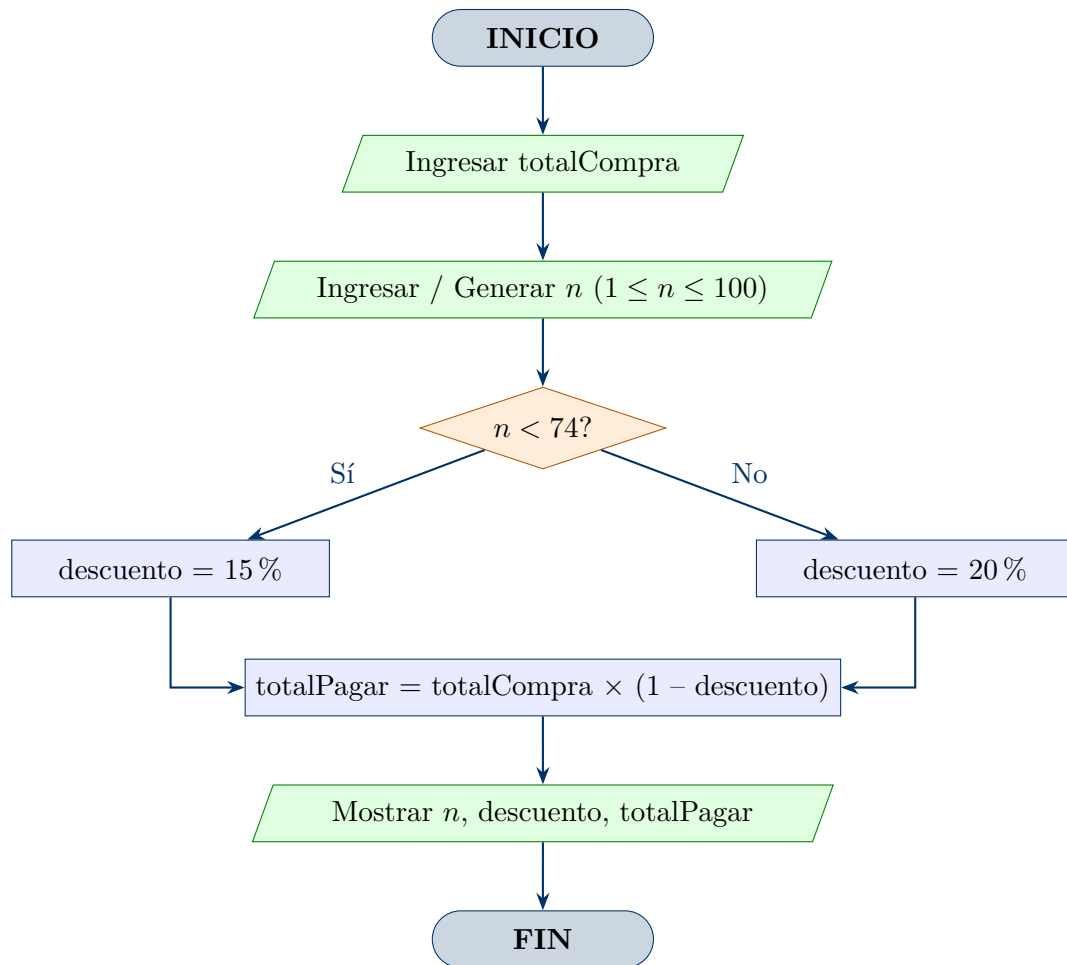


Figura 5: Diagrama de flujo – Ejercicio 10: Descuento en Supermercado

4.5.3 Pseudocódigo

Algoritmo 5 Descuento en Supermercado**Require:** $totalCompra > 0$ **Ensure:** $totalPagar$, $descuentoPct$

```

1: LEER  $totalCompra$ 
2: LEER o GENERAR  $n$  tal que  $1 \leq n \leq 100$ 
3: if  $n < 74$  then
4:    $descuento \leftarrow 0,15$ 
5: else
6:    $descuento \leftarrow 0,20$ 
7: end if
8:  $montoDescuento \leftarrow totalCompra \times descuento$ 
9:  $totalPagar \leftarrow totalCompra - montoDescuento$ 
10: MOSTRAR  $n$ ,  $descuento \times 100 \%$ ,  $montoDescuento$ ,  $totalPagar$ 

```

4.5.4 Implementación MVC

- **Modelo** (`supermercado_model.dart`): método estático `calcular(totalCompra, numeroAzar)` que retorna un objeto con todos los valores calculados (porcentaje, monto descontado, total a pagar).
- **Controlador** (`supermercado_controller.dart`): método `procesar()` valida las entradas, genera el número aleatorio con `dart:math` si se omite, y formatea el resultado.
- **Vista** (`supermercado_view.dart`): dos campos de entrada (total y número opcional) y un área de resultados con formato de moneda.

5. Conclusiones

1. La adopción del patrón **MVC** en Flutter permite una separación clara de responsabilidades: los modelos contienen lógica de negocio pura y son completamente probables de forma unitaria, los controladores gestionan el estado reactivo mediante `ChangeNotifier`, y las vistas permanecen desacopladas de cualquier lógica de cálculo.
2. La comparación con el **Diseño Atómico** evidencia que ambos paradigmas son ortogonales y complementarios: MVC organiza la arquitectura a nivel de archivos y módulos funcionales, mientras que el Diseño Atómico guía la composición interna de los widgets dentro de cada Vista, promoviendo la reutilización de componentes de interfaz.
3. **Flutter y Dart** demostraron ser una plataforma eficiente para el desarrollo rápido de aplicaciones multiplataforma. El sistema de widgets composables y el *Hot Reload* aceleran significativamente el ciclo de desarrollo, mientras que *null safety* mejora la robustez del código.
4. La organización modular del proyecto (una carpeta por capa) mejoró la mantenibilidad y facilitó la colaboración entre los integrantes del equipo, ya que cada desarrollador podía trabajar en su capa asignada con mínimas dependencias hacia los demás.
5. Los cinco ejercicios implementados cubrieron escenarios de cálculo con lógica condicional, iteración y conversión de unidades, lo que permitió al equipo aplicar los fundamentos del desarrollo móvil en situaciones prácticas y verificar la correctitud de las implementaciones.

6. Recomendaciones

1. **Incorporar pruebas unitarias automatizadas:** se recomienda complementar las pruebas manuales realizadas con una suite de pruebas unitarias usando el paquete `flutter_test`. Dado que los modelos son clases Dart puras sin dependencias de Flutter, su cobertura con pruebas unitarias es directa y de bajo costo, lo que incrementaría significativamente la confiabilidad del código ante cambios futuros.
2. **Adoptar un gestor de estado más robusto:** para proyectos de mayor escala, se sugiere migrar de `ChangeNotifier` a soluciones más estructuradas como **Riverpod** o **Bloc**, que ofrecen mejor soporte para pruebas, mayor trazabilidad del estado y separación más estricta entre la lógica de negocio y la capa de

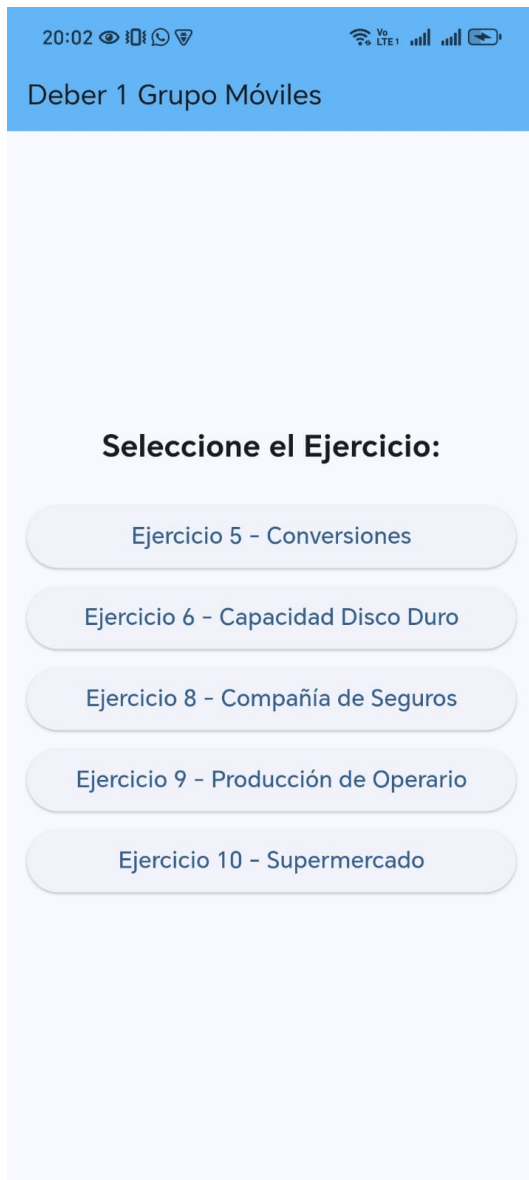
presentación.

3. **Implementar validación de entradas en el modelo:** actualmente la validación de datos se realiza parcialmente en los controladores. Se recomienda centralizar las reglas de validación dentro de los modelos o en clases de validación dedicadas, siguiendo el principio de responsabilidad única (*Single Responsibility Principle*) y facilitando su reutilización y prueba independiente.
4. **Agregar internacionalización (*i18n*):** la aplicación muestra mensajes y etiquetas únicamente en español. Para escalar el proyecto a audiencias más amplias, se recomienda implementar el soporte de internacionalización mediante el paquete `flutter_localizations`, lo que permitiría añadir nuevos idiomas sin modificar la lógica existente.
5. **Mejorar la accesibilidad:** se recomienda revisar el árbol de widgets para garantizar el cumplimiento de las pautas de accesibilidad, en particular: asignar etiquetas semánticas a los campos de entrada mediante `Semantics`, verificar el contraste de colores según WCAG 2.1 y asegurar que la aplicación sea completamente navegable con lectores de pantalla (*TalkBack* en Android, *VoiceOver* en iOS).
6. **Publicar y configurar un *pipeline* de CI/CD:** para facilitar la colaboración y la entrega continua, se recomienda configurar un flujo de integración y despliegue continuo (CI/CD) con **GitHub Actions** o **Codemagic**, que ejecute automáticamente las pruebas, el análisis estático (`flutter analyze`) y la generación del APK de release ante cada *pull request*.

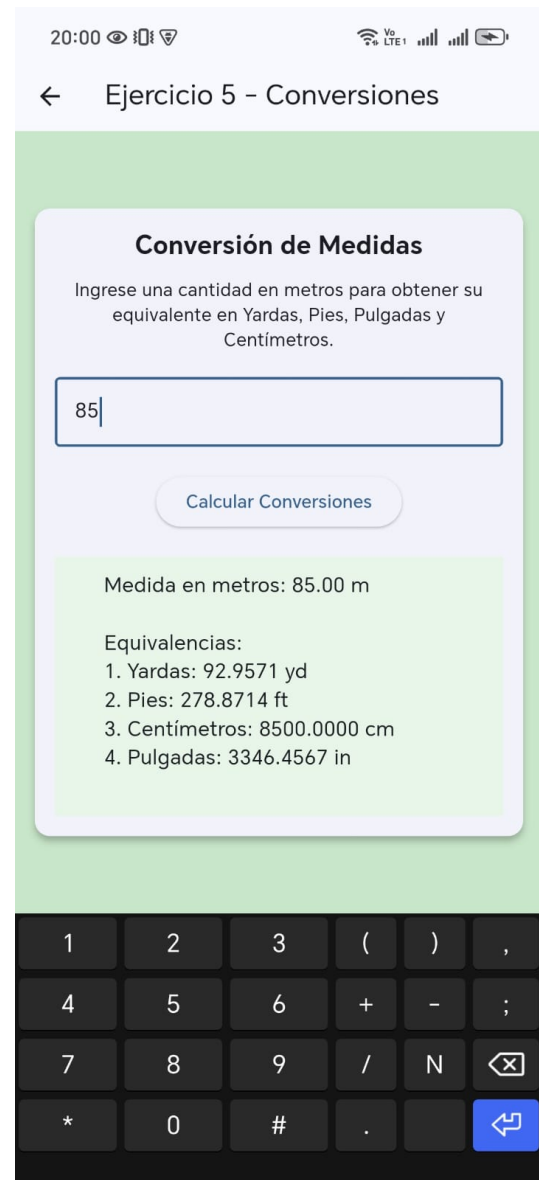
Referencias

- [1] Google LLC. (2024). *Flutter – Build apps for any screen*. Flutter Documentation. Recuperado de <https://flutter.dev/docs>
- [2] Google LLC. (2024). *Dart programming language*. Dart Documentation. Recuperado de <https://dart.dev/guides>
- [3] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. ISBN: 978-0-201-63361-0.
- [4] Frost, B. (2016). *Atomic Design*. Brad Frost Web. Recuperado de <https://atomicdesign.bradfrost.com>
- [5] Google LLC. (2024). *State management – Simple app state management*. Flutter Documentation. Recuperado de <https://docs.flutter.dev/data-and-backend/state-mgmt/simple>
- [6] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. ISBN: 978-0-13-468599-1.
- [7] Windmill, E. (2020). *Flutter in Action*. Manning Publications. ISBN: 978-1-61729-571-8.
- [8] Seiden, M. (2023). *Pragmatic Flutter: Building Cross-Platform Mobile Apps for Android, iOS, Web & Desktop*. Apress. ISBN: 978-1-4842-6195-3.

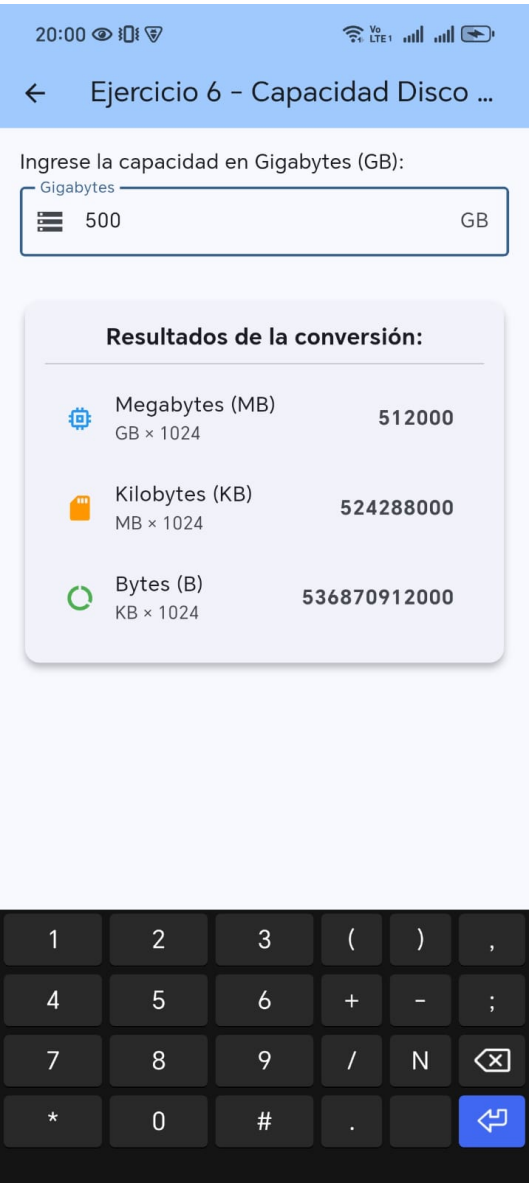
Anexos



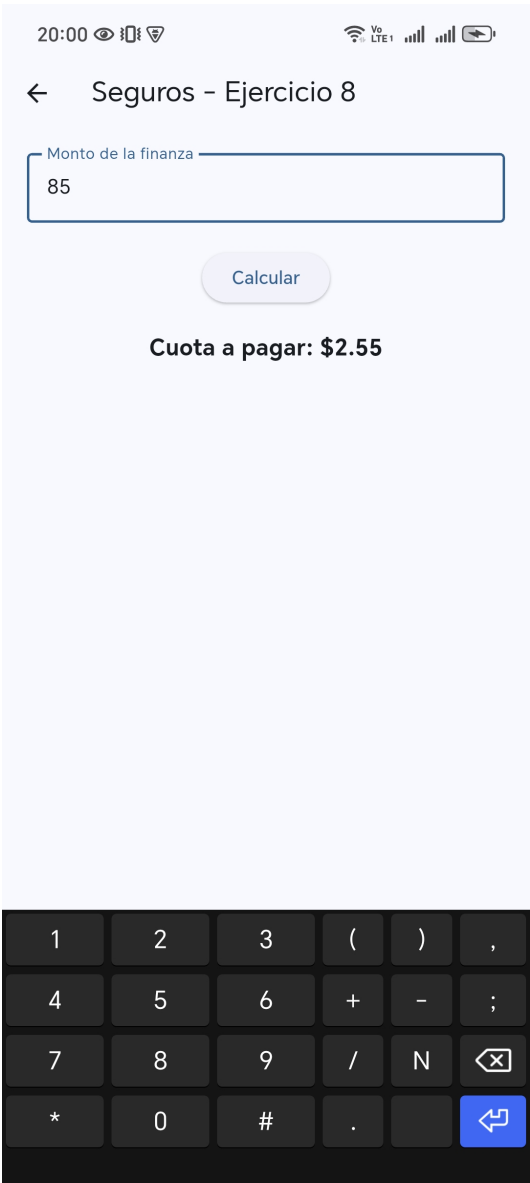
Anexo 1: Menú principal de la aplicación



Anexo 2: Ejercicio 5 – Conversión de Medidas



Anexo 3: Ejercicio 6 – Capacidad de Disco Duro



Anexo 4: Ejercicio 8 – Compañía de Seguros

20:01

← Incentivos por Producción

Lunes	Martes
5	12

Miércoles	Jueves
85	9

Viernes	Sábado
56	5

Calcular Incentivo

Promedio semanal: 28.67 unidades.
No recibirá incentivos.

Anexo 5: Ejercicio 9 – Producción de Operario

20:01

← Ejercicio 10 - Supermercados

Promoción del Supermercado

¡Descuento sorpresa! Si su número es menor a 74, gana 15%. Si es 74 o mayor, gana 20%.

800

6

Calcular Descuento

Total de la compra: \$800.00
Número escogido: 6
Porcentaje de descuento: 15%
Dinero descontado: \$120.00
Total a pagar: \$680.00

1 2 3 () ,
4 5 6 + - ;
7 8 9 / N ✕
* 0 # . ↩

Anexo 6: Ejercicio 10 – Supermercado